

Indian Statistical Institute
BSDS Semester-I 2026-2027
Mid-semester Examination (30 September, 2026)
Subject: Introduction to Computing
Total: 25 marks Duration: 1 hr.

PART A: This part of the exam is a conventional, closed-book exam.

1. Suppose the following statements are executed one by one at the Python prompt. For each line containing a call to `print()`, state what output will be generated, or whether an error message will be printed. You do not need to copy the statements; just the statement number and the output will suffice. Provide a very brief justification (1-2 lines) for each answer. [17]

1 <code>s = "3.1415926e2"</code>	12 <code>l = ["x", "y", "z", "xy", "yz", "zx", "xyz"]</code>
2 <code>print(int(s))</code>	13 <code>print(sorted(l))</code>
3 <code>print(int(float(s)))</code>	14 <code>l = [2, 10, 20, 1]</code>
4 <code>print(bool(None))</code>	15 <code>print(sorted(l))</code>
5 <code>print(bool(-100))</code>	16 <code>print(l)</code>
6 <code>print(bool(-0))</code>	17 <code>l.sort()</code>
7 <code>x = 2</code>	18 <code>print(l)</code>
8 <code>print("Exam is for " + x + " hours")</code>	19 <code>print(sorted(["1", "2", "10", "20"]))</code>
9 <code>print("Exam is for ", x, " hours")</code>	20 <code>y = 3</code>
10 <code>print(x * "BSDS")</code>	21 <code>x, y = y, x+y</code>
11 <code>print("x" * "BSDS")</code>	22 <code>print(x, y)</code>

2. (a) Without using the `itertools` library, write a function `cartesian_product(L1, L2)` that takes two lists `L1` and `L2` as input, and prints all pairs (X, Y) where X is an element of `L1`, and Y is an element of `L2`.
(b) If `L1` is `["a", 5.9, (1,2)]` and `L2` is `[[3.14, 159], "hi", (7,7)]`, what output will `cartesian_product(L1, L2)` produce? [2 + 1 = 3]

3. The following function `reverse_in_place(L)` is supposed to reverse a list `L` *in place*, without using the `reverse()` or `reversed` library functions. Fill in the blanks so that it works as desired. You do not need to copy the function; just write the part number, and what you would write in the corresponding blank. Very briefly explain your reasoning (5 lines or less). [5]

```
def reverse_in_place(L):  
    for i in ____ (a) ____ :  
        L[ ____ (b) ____ ], L[ ____ (c) ____ ] = L[ ____ (d) ____ ], L[ ____ (e) ____ ]  
    print(L)  
    return L
```

Indian Statistical Institute
BSDS Semester-I 2026-2027

Mid-semester Examination (30 September, 2026)

Subject: Introduction to Computing

Total: 30 marks

Maximum marks: 25

Duration: 1 hr.

PART B: This part of the exam is an open-book, open-notes exam.

You do not need to copy any code provided in the question paper. Just write the code for the missing parts, along with the corresponding part number, and a brief justification for your answer.

1. The figure below shows the letter Y drawn on the terminal using the # character.

```
###      gap      ###
###      stroke_width
###      ###
###      ###
###      ###
#####
#####
#####
#####
#####
#####
#####
```

An outline of a function `print_y_pattern` is given below. Fill in the blanks in the body of the function so that it draws a figure on the terminal (like the one shown above), using the values of the 3 parameters (dimensions) shown in the figure. One line and one column of the terminal occupy unit length in the vertical and horizontal direction, respectively; thus, in the above example, `stroke_width` is 3, `gap` is 8, and `height` is 6 (you may assume that `gap` is an even number).

[8]

```
def print_y_pattern(stroke_width, gap, height):
    for i in ____ (a) ____:
        print(' ' * ____ (b) ____ +
              '#' * ____ (c) ____ +
              ' ' * ____ (d) ____ +
              '#' * ____ (e) ____ )
    for i in ____ (f) ____:
        print(' ' * ____ (g) ____ +
              '#' * ____ (h) ____ )
```

2. *Hangman* is a game played by two players. One of the players *A* thinks of a word (say *w*), consisting of *m* letters, and presents it to the other player *B*, with some/all letters missing. *B* tries to guess the word, one letter at a time. If the letter guessed by *B* occurs in *w*, *A* fills in **all** the positions in *w* where the letter occurs; otherwise, *B* loses a 'life'. The process continues, until *B* correctly guesses the word, or runs out of lives.

An outline of a function `hangman(word, guesses, lives)` is given below. It takes two strings and an integer as parameters (the parameters are supposed to have self-explanatory names), displays the progress of the game, and finally prints whether

- *B* has correctly guessed the word (in this case, if there are any extra letters in `guesses`, they are ignored);
- *B* has run out of lives; or
- the game is incomplete.

Fill in the blanks in the body of the function so that it works as intended. For example, if the function is called as `hangman("lion", "euiaoblnxy", 6)`, it should print the following. [10]

```

_ _ _ _      6 lives remaining
_ _ _ _      "e" guessed, 5 lives remaining
_ _ _ _      "u" guessed, 4 lives remaining
_ i _ _      "i" guessed, 4 lives remaining
_ i _ _      "a" guessed, 3 lives remaining
_ i o _      "o" guessed, 3 lives remaining
_ i o _      "b" guessed, 2 lives remaining
l i o _      "l" guessed, 2 lives remaining
l i o n      "n" guessed, 2 lives remaining

```

Word correctly guessed.

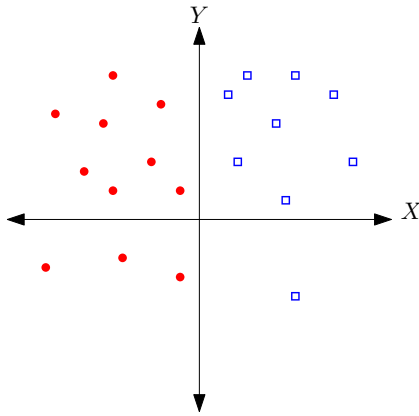
```

def hangman(word, guesses, lives):
    remaining_letters = set(____(a)____)
    current_status = ['_ ' for i in ____ (b) ____]
    for c in guesses:
        if c in remaining_letters:
            ____ (c) ____
            for i, l in enumerate(word):
                if ____ (d) ____:
                    ____ (e) ____
        else:
            ____ (f) ____

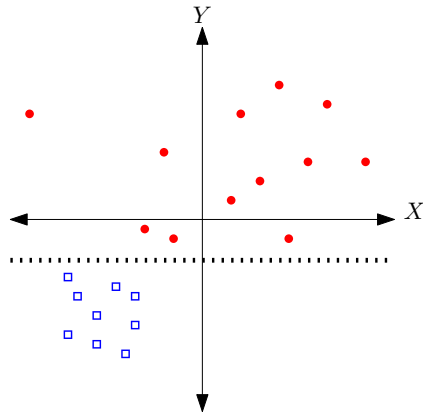
    print(''.join(current_status), '\t', c, 'guessed,', lives, 'lives remaining')
    if ____ (g) ____:
        print('Word correctly guessed')
        return
    elif ____ (h) ____:
        print('Game over')
        return
    print('Game incomplete')
    return

```

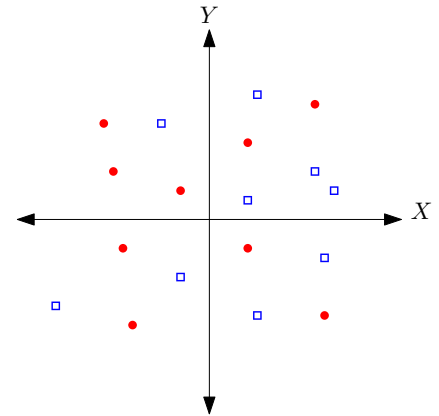
3. You are given the coordinates of some points in the X - Y plane. $N_R > 0$ of the points are coloured red (denoted by dots in the figures below), while the remaining $N_B > 0$ points are coloured blue (denoted by squares in the figures below). The following program is supposed to determine whether the red and blue points can be separated by a horizontal and/or vertical line. Fill in the blanks so that the program works as intended. [12]



The red and blue points can be separated by the Y -axis (and infinitely many other lines parallel to the Y -axis).



The red and blue points can be separated by the dotted line, which is parallel to the X -axis.



The red and blue points cannot be separated by any line that is parallel to the X or the Y -axis.

Input format: If there are N points, the input data will consist of N lines, each of which will contain 3 space-separated fields: 2 floating point numbers corresponding to the X and Y coordinates of the point, and a single letter, R or B, denoting the colour of the point.

```
min_red_x = max_red_x = min_red_y = max_red_y = None
min_blue_x = max_blue_x = min_blue_y = max_blue_y = None
while True:
    s = input()
    if s == "": # To end the input, the user hits ENTER without typing any data
        break
    x, y, colour = _____ (1)
    x = _____ (2)
    y = _____ (3)
    match colour:
        case 'R':
            if min_red_x == None:
                _____ (4)
            min_red_x = _____ (5)
            max_red_x = _____ (6)
            min_red_y = _____ (7)
            max_red_y = _____ (8)
```

```
case 'B':
    if min_blue_x == None:
        _____(9)_____
    min_blue_x = _____(10)_____
    max_blue_x = _____(11)_____
    min_blue_y = _____(12)_____
    max_blue_y = _____(13)_____
if _____(14)_____:
    print("Points can be separated by a line parallel to the Y axis")

if _____(15)_____:
    print("Points can be separated by a line parallel to the X axis")
```